

EXPERIMENT-8: Session tracking with cookies and HTTP sessions.

Explore session tracking mechanisms like cookies and HTTP sessions to maintain user transaction history.

Objective : Implement persistence of user state across requests using session techniques.

Tools : Cookies, HTTP Session, server-side state.

Outcome : A web application that preserves user transaction history using sessions and cookies.

Answer:

Use cookies or session storage to keep the user's cart and login state across page visits.

File Name: session_app.js

Code:

```
// =====  
// Experiment: Session Tracking with Cookies and HTTP Sessions  
// Shopping app: Login + Cart + Transaction History  
// Run: node session_app.js   Open: http://localhost:3000  
// No npm install needed (built-in modules only)  
// =====  
  
const http = require('http');  
const crypto = require('crypto');  
  
// ----- DATA -----  
const PRODUCTS = [  
  { id: 1, name: 'Book', price: 300 },  
  { id: 2, name: 'Pen', price: 20 },  
  { id: 3, name: 'Notebook', price: 80 },  
  { id: 4, name: 'Bag', price: 900 },  
];  
  
// Server-side session store (like HttpSession): sessionId -> data  
const sessions = new Map();  
const TIMEOUT = 10 * 60 * 1000; // 10 minutes of inactivity  
  
// ----- COOKIE HELPERS -----
```

```

function parseCookies(req) {
  const out = {};
  (req.headers.cookie || '').split(';').forEach(p => {
    const i = p.indexOf('=');
    if (i > 0) out[p.slice(0, i).trim()] = decodeURIComponent(p.slice(i + 1).trim());
  });
  return out;
}

```

```

function setCookie(res, name, value, attrs = '') {
  const prev = res.getHeader('Set-Cookie') || [];
  res.setHeader('Set-Cookie',
    [].concat(prev, `${name}=${encodeURIComponent(value)}; Path=/; ${attrs}`));
}

```

// ----- SESSION HELPERS -----

```

function getSession(req) {
  const sid = parseCookies(req).SID;
  const s = sid && sessions.get(sid);
  if (!s) return null;
  if (Date.now() - s.lastAccess > TIMEOUT) { sessions.delete(sid); return null; }
  s.lastAccess = Date.now();
  return s;
}

```

```

function createSession(res, user) {
  const id = crypto.randomUUID();
  const s = { id, user, cart: {}, history: [], visits: 0, created: new Date(), lastAccess: Date.now() };
  sessions.set(id, s);
  setCookie(res, 'SID', id, 'HttpOnly'); // session cookie (deleted when browser closes)
  return s;
}

```

// ----- GENERAL HELPERS -----

```

const esc = s => String(s).replace(/<|>|"/g,
  c => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' }[c]));

```

```
function readBody(req) {
  return new Promise(resolve => {
    let d = '';
    req.on('data', c => (d += c));
    req.on('end', () => resolve(Object.fromEntries(new URLSearchParams(d))));
  });
}
```

```
function redirect(res, msg = '') {
  res.writeHead(303, { Location: '/' + (msg ? '?msg=' + encodeURIComponent(msg) : '') });
  res.end();
}
```

```
const style = `<style>
  body{font-family:Arial;max-width:850px;margin:30px auto;padding:0 15px}
  table{border-collapse:collapse;width:100%;margin:10px 0}
  th,td{border:1px solid #ccc;padding:7px;text-align:left}
  th{background:#2c3e50;color:#fff}
  input,button{padding:6px 10px;margin:2px}
  .msg{background:#e8f5e9;padding:8px;border-left:4px solid green;margin:10px 0}
  .box{background:#f4f6f8;padding:10px;border-radius:6px;margin:10px 0;font-size:14px}
</style>`;
```

// ----- VIEWS -----

```
function loginPage(lastUser, msg) {
  return `<!DOCTYPE html><html><head><meta charset="UTF-
    8"><title>Login</title>${style}</head><body>
    <h1>🛒 Shopping App - Login</h1>
    ${msg ? `<div class="msg">${esc(msg)}</div>` : ''}
    <form method="POST" action="/">
      <input type="hidden" name="action" value="login">
      <input name="user" placeholder="Username" required value="${esc(lastUser)}">
      <button>Login</button>
    </form>
```

```

    <p class="box">Username is pre-filled from the <b>lastUser</b> cookie (persistent cookie, 30
      days).</p>
  </body></html>`;
}

```

```

function shopPage(s, lastVisit, msg) {
  const pRows = PRODUCTS.map(p => `<tr><td>${p.name}</td><td>₹${p.price}</td><td>
    <form method="POST" action="/" style="display:inline">
      <input type="hidden" name="action" value="addToCart">
      <input type="hidden" name="id" value="${p.id}">
      <button>Add to Cart</button></form></td></tr>`).join("");

  let total = 0;

  const cRows = Object.entries(s.cart).map(([id, qty]) => {
    const p = PRODUCTS.find(x => x.id == id);
    total += p.price * qty;
    return `<tr><td>${p.name}</td><td>${qty}</td><td>₹${p.price * qty}</td><td>
      <form method="POST" action="/" style="display:inline">
        <input type="hidden" name="action" value="removeFromCart">
        <input type="hidden" name="id" value="${id}">
        <button>Remove</button></form></td></tr>`;
  }).join("");

  const hRows = s.history.map((h, i) => `<tr><td>${i + 1}</td><td>${h.time}</td>
    <td>${h.items.map(x => `${x.name} x${x.qty}`).join(', ')}</td><td>₹${h.total}</td></tr>`).join("");

  return `<!DOCTYPE html><html><head><meta charset="UTF-
    8"><title>Shop</title>${style}</head><body>
    <h1>🛒 Welcome, ${esc(s.user)}</h1>
    <form method="POST" action="/" style="display:inline">
      <input type="hidden" name="action" value="logout"><button>Logout</button></form>
    ${msg ? `<div class="msg">${esc(msg)}</div>` : ""}

    <div class="box">
      <b>Session ID:</b> ${s.id.slice(0, 8)}... (stored on server, only ID is in cookie)<br>
      <b>Visits in this session:</b> ${s.visits}<br>

```

```

<b>Session created:</b> ${s.created.toLocaleTimeString()}<br>
<b>Last visit (from cookie):</b> ${esc(lastVisit || 'First visit')}
</div>

```

```

<h2>Products</h2>

```

```

<table><tr><th>Name</th><th>Price</th><th>Action</th></tr>${pRows}</table>

```

```

<h2>Your Cart (saved in session)</h2>

```

```

<table><tr><th>Item</th><th>Qty</th><th>Subtotal</th><th>Action</th></tr>

```

```

  ${cRows || '<tr><td colspan="4">Cart is empty</td></tr>'}</table>

```

```

<form method="POST" action="/">

```

```

  <input type="hidden" name="action" value="checkout">

```

```

  <b>Total: ₹${total}</b> <button ${total ? '' : 'disabled'}>Checkout</button></form>

```

```

<h2>Transaction History (saved in session)</h2>

```

```

<table><tr><th>#</th><th>Time</th><th>Items</th><th>Total</th></tr>

```

```

  ${hRows || '<tr><td colspan="4">No transactions yet</td></tr>'}</table>

```

```

</body></html>`;

```

```

}

```

```

// ----- CONTROLLER -----

```

```

function doGet(req, res) {

```

```

  const cookies = parseCookies(req);

```

```

  const msg = new URL(req.url, 'http://x').searchParams.get('msg') || '';

```

```

  const s = getSession(req);

```

```

  res.setHeader('Content-Type', 'text/html; charset=utf-8');

```

```

  if (!s) { res.end(loginPage(cookies.lastUser || '', msg)); return; }

```

```

  s.visits++;

```

```

  // Persistent cookie: remember last visit time for 30 days

```

```

  setCookie(res, 'lastVisit', new Date().toLocaleString(), 'Max-Age=2592000');

```

```

  res.end(shopPage(s, cookies.lastVisit, msg));

```

```

}

```

```

async function doPost(req, res) {

```

```

const f = await readBody(req);
const s = getSession(req);

if (f.action === 'login') {
  const user = (f.user || '').trim();
  if (!user) return redirect(res, 'Enter a username');
  createSession(res, user);
  setCookie(res, 'lastUser', user, 'Max-Age=2592000'); // persistent cookie
  return redirect(res, 'Logged in successfully');
}

if (!s) return redirect(res, 'Session expired. Please login again.');
```

```

switch (f.action) {
  case 'addToCart':
    s.cart[f.id] = (s.cart[f.id] || 0) + 1;
    return redirect(res, 'Item added to cart');
  case 'removeFromCart':
    delete s.cart[f.id];
    return redirect(res, 'Item removed');
  case 'checkout': {
    const items = Object.entries(s.cart).map(([id, qty]) => {
      const p = PRODUCTS.find(x => x.id == id);
      return { name: p.name, qty, price: p.price };
    });
    if (!items.length) return redirect(res, 'Cart is empty');
    const total = items.reduce((t, i) => t + i.price * i.qty, 0);
    s.history.push({ time: new Date().toLocaleString(), items, total });
    s.cart = {};
    return redirect(res, 'Purchase completed! Added to history.');
```

```

  }
  case 'logout':
    sessions.delete(s.id);
    setCookie(res, 'SID', '', 'Max-Age=0'); // delete the cookie
    return redirect(res, 'Logged out');
  default:

```

```
    return redirect(res, 'Unknown action');
  }
}

// ----- SERVER -----
http.createServer(async (req, res) => {
  try {
    if (req.method === 'GET') doGet(req, res);
    else if (req.method === 'POST') await doPost(req, res);
    else { res.writeHead(405); res.end('Method Not Allowed'); }
  } catch (e) {
    console.error(e);
    res.writeHead(500); res.end('Server error');
  }
}).listen(3000, () => console.log('Server running at http://localhost:3000'));
```